# Kafka & Redis Study Guide

What to search for on YouTube, mapped to exactly where it's used (or will be used) in this project. Kafka is already built and wired in, just never run against a real broker yet. Redis hasn't been started at all.

---

## Kafka — already built, just never run against a real broker

| Concept to search | Where it's used here |
|---|---|
| **Producer / Consumer / Topic / Broker / Partition** — the core vocabulary | Foundational — everything below builds on this |
| `KafkaTemplate<K,V>` — how Spring sends messages | `TransactionEventProducer`, `BeneficiaryEventProducer` |
| `@KafkaListener` — how Spring receives messages | `TransactionEventConsumer`, `BeneficiaryEventConsumer` (in `notification-service`) |
| **Message key & partitioning** — why the same key always lands in the same partition, and why that preserves ordering | We key by `accountId` / `customerId` — search "Kafka partition key ordering guarantee" |
| **Serializers / Deserializers** (`StringSerializer`, and why we manually JSON-encode via Jackson instead of a typed Kafka serializer) | `spring.kafka.producer.key-serializer` / `value-serializer` in `application.properties` |
| `ProducerFactory` / `ConsumerFactory` — why Spring Boot's auto-configured `KafkaTemplate` bean didn't satisfy our typed injection, and why we hand-wrote `KafkaProducerConfig` | A real bug we hit — good one to understand deeply |
| **Consumer groups** (`group-id`) — how multiple consumer instances split work | `spring.kafka.consumer.group-id=notification-service` |
| `auto-offset-reset` (`earliest` vs `latest`) | Set in `notification-service`'s `application.yml` |
| **Delivery semantics** — at-least-once vs exactly-once, and why that means a consumer might see the same message twice | Directly explains *why* we built idempotency keys |

| Concept to search | Where it's used here |
|---|---|
| **Idempotent consumption** | `event.eventId()` is passed as `idempotencyKey` into `NotificationRequest` — `notification-service` has a unique DB constraint on it specifically so a redelivered message never double-notifies |

**Not built yet, but on your own roadmap (Days 58–61)** — worth knowing these terms exist even before you get there: **dead-letter queue**, **retry topics**, **consumer lag**, **manual vs auto ack**.

---

## Redis — next up, Days 56–57, nothing written yet

| Concept to search | Where it will be used here |
|---|---|
| **Key-value store basics** — what Redis actually is vs a relational DB | Foundational |
| **TTL / expiration** — keys that auto-delete after N seconds | Core to caching — an account's cached data shouldn't live forever |
| **Cache-aside pattern** — check cache first, fall through to DB on a miss, write result back to cache | The actual strategy you'll implement for Account lookups |
| `spring-boot-starter-data-redis`, `RedisTemplate`, Lettuce (the default Redis client Spring Boot uses) | Day 56 — Redis configuration |
| **Spring's Cache abstraction**: `@EnableCaching`, `@Cacheable`, `@CacheEvict`, `@CachePut`, `CacheManager` / `RedisCacheManager` | Day 57 — this is almost certainly the approach you'll want (declarative, not manual `RedisTemplate` calls) |
| **Cache invalidation** — the famous "there are only two hard problems in computer science" joke exists for a reason | Critical here: `AccountServiceImpl.getAccountById()` / `getAccountByAccountNumber()` are perfect `@Cacheable` candidates (read often, don't change often) — but `deposit()`, `withdraw()`, `transfer()`, `freezeAccount()`, `closeAccount()` all change the balance/status, so each needs a matching `@CacheEvict` or the cache goes stale and shows wrong balances |

The one thing worth internalizing before you touch code: **a cached account balance is dangerous if you forget to evict it on every single write path.** That's the real risk in Day 57, more than the

Redis mechanics themselves.